

第十二次作业反馈

魏远毅

2024 年 12 月 10 日

1 问题 8.1

1.1 问题描述

以关键字序列 (5,1,6,0,9,2,8,3,7,4) 为例，手动执行下列排序算法，写出每一趟排序结束时的关键字序列状态。

(1) 直接插入排序 (2) 希尔排序 (取增量为 5,3,1)

(3) 快速排序 (4) 冒泡排序

(5) 归并排序 (6) 堆排序

1.2 解答

(1) 直接插入排序：(左边有序，右边无序)

5|160928374

15|60928374

156|0928374

0156|928374

01569|28374

012569|8374

0125689|374

01235689|74

012356789|4

0123456789|

(2) 希尔排序

5160928374

增量为 5:

2160958374(5、2 一组)

2160958374(1、8 一组)

2130958674(6、3 一组)

2130958674(0、7 一组)

2130458679(9、4 一组)

增量为 3:

0132458679(2、0、8、9 一组)

0132458679(1、4、6 一组)

0132458679(3、5、7 一组)

增量为 1:

0123456789(0、1、2、3、4、5、6、7、8、9 一组)

(3) 快速排序 (选择第一个元素为中枢)

10234|5|8976(5 为中枢)

0|1|234|5|8976(1 为中枢)

0|1|234|5|76|8|9(8 为中枢)

0|1|234|5|6|7|8|9(7 为中枢)

(4) 冒泡排序

5160928374

150628374|9(第一轮冒泡)

10526374|89(第二轮冒泡)

0125364|789(第三轮冒泡)

012354|6789(第四轮冒泡)

01234|56789(第五轮冒泡)

(5) 归并排序

5160928374

1560928374(第一层归并)

1560923847(第二层归并)

0156923478(第三层归并)

012356789(第四层归并)

(6) 堆排序 (采用大根堆)

5160928374

9785426301(9 为堆顶)

876542130|9(8 为堆顶)

75634210|89(7 为堆顶)

6523401|789(6 为堆顶)

542310|6789(5 为堆顶)

43201|56789(4 为堆顶)

3120|456789(3 为堆顶)

210|3456789(2 为堆顶)

10|23456789(1 为堆顶)

0123456789

2 问题 8.4

2.1 题目描述

10000 人参加比赛，录取成绩优异的前十名，将他们从高到低输出，请问那种排序方法速度最快，为什么？

2.2 解答

这是非常经典的 Top-k 问题：查找数组中前 k 个最大或最小的元素。

下面我们讨论几种算法思路及其时间复杂度：

1. 全局排序：

使用快速排序等排序算法进行全局排序，时间复杂度为 $O(n\log n)$

2. 局部排序：

可以使用冒泡排序算法，只需要进行 k 轮冒泡我们就可以得到前 k 个最大或最小的元素；

一轮冒泡的时间复杂度为 $O(n)$ ，所以总时间复杂度为 $O(k * n)$

3. 堆排序：

将数组的前 k 个元素构建为小根堆；然后遍历后面 $n-k$ 个元素，比较他们与堆顶的大小，如果大于堆顶，则与堆顶交换并维护小根堆；最后小根堆中的元素就是最大的 k 个元素；

最坏情况下，总是要维护小根堆且每次维护的时间开销为 $O(\log k)$ ，那么总时间复杂度为 $O(n\log k)$

2.3 拓展问题

Top-k 问题的一种变体：查找数组中第 k 大或小的元素。

这里介绍快速排序算法的一种变体：快速选择算法

在快速排序算法中，我们会进行数组的分割 partition，然后对中枢两边都进行递归排序；

但是当我们的目标是找出第 k 大的元素时，其实我们只需要中枢的一侧进行递归查找；

比如长度为 n 的数组，在中枢分割后，左侧长度为 m ，右侧长度为 $n-m$ 且 $n-m > k$ ，那么自然而然第 k 大的元素一定在中枢右侧，后面我们只需要递归右侧数组即可。

该算法的时间复杂度为 $O(n)$ ，相比于多次冒泡快不少。

该算法改进后同样可以用于经典 Top-k 问题，主要还是分而治之的算法思想，这里就不介绍了，感兴趣的同学可以自己查一下。

3 问题 8.13

假设输入序列中仅有三种颜色：R, W, B，请写一个算法将序列按照 R, W, B 的顺序排好。

3.1 解答

这道题几乎所有同学都采用了计数排序的方法，该方法比较简单，这里就不介绍了。

(注：我刚开始改的时候没有考虑计数排序，部分同学的作业上改错了)

在实际编程中，我们往往会面对不是数字但需要排序的数据，比如该题目中的色彩。

对此，我们通常会设定一个优先级比较函数来取代数字比较中的 $>$ 、 $<$ 等比较操作。

```
1 bool compare(char a, char b) {  
2     if (a==b) return false;  
3     if (a=='R') return true;  
4     if (a=='W' && b=='B') return true;  
5     return false;  
6 }
```

这样我们就可以用基于数字大小的排序算法来对字符串进行排序操作。

在 C++ 中，我们常会利用这种自定义的优先级比较函数来定义一个比较自由的优先队列。